45

数据完整性与保护

除了前面研究过的文件系统（file system）中的那些基本进展，还有不少特性值得研究。本章再次关注可靠性（reliability），存储系统的可靠性此前在 RAID 一章已经研究过。具体来说，鉴于现代存储设备（storage device）并不可靠，文件系统或存储系统应该如何确保数据安全?

这个领域统称为**数据完整性**（data integrity）或**数据保护**（data protection）。下面就来研究一些技术，确保你存进存储系统的数据，取回来时还是原来的样子。

关键问题：如何确保数据完整性
系统应该如何确保写入存储的数据受到保护?需要哪些技术?如何让这些技术高效，空间和时间开销都很低?

45.1 磁盘故障模式

你在 RAID 一章中已经了解到，磁盘并不完美，偶尔会出故障。在早期的 RAID 系统中，故障模型相当简单：要么整块磁盘正常工作，要么彻底失效，而且检测这种故障很直接。这种 **fail-stop** 磁盘故障模型让构建 RAID 变得相对简单 [S90]。

但你还没了解现代磁盘的其他各种故障模式。正如 Bairavasundaram 等人详细研究过的那样 [B+07, B+08]，现代磁盘偶尔会看起来大体正常，却无法成功访问一个或多个块（block）。具体来说，有两类单块故障很常见，值得考虑：**latent sector error**（**LSE**）和**块损坏**（block corruption）。下面我们分别详细讨论。

LSE 是指磁盘扇区（sector）或一组扇区以某种方式受损。例如，如果磁头（disk head）因为某种原因碰到了盘面（surface），也就是发生了正常运行时本不应出现的**磁头碰撞**（head crash），就可能损伤盘面，导致这些位无法读取。宇宙射线也会让位发生翻转，导致内容出错。幸运的是，驱动器内部使用了**纠错码**（error correcting code，**ECC**），驱动器用它判断

	廉价	高价
LSE	9.40%	1.40%
损坏	0.50%	0.05%

图 45.1：**LSE 和块损坏的发生频率**

一个块在盘上的各个位是否完好，有时还能修复它们；如果数据位有问题，而驱动器又没有足够的信息来纠正错误，那么请求读取这些位时，磁盘就会返回错误。

还有一些情况下，磁盘块会以磁盘本身检测不到的方式**损坏**（corrupt）。例如，有 bug 的磁盘固件（firmware）可能把一个块写到错误的位置；在这种情况下，磁盘的 ECC 显示块的内容没问题，但从客户端的角度看，之后访问时返回的却是错误的块。类似地，块从主机经过有故障的总线（bus）传到磁盘时，也可能被损坏；磁盘存下的是损坏后的数据，但这并不是客户端想要的。这类故障格外隐蔽，因为它们是**静默错误**（silent fault）：磁盘返回有问题的数据时，没有给出任何出了问题的迹象。

Prabhakaran 等人把这种更现代的磁盘故障观点描述为 **fail-partial** 磁盘故障模型 [P+05]。按照这种观点，磁盘仍然可能整体失效（就像传统的 fail-stop 模型那样）；但磁盘也可能看起来在正常工作，却有一个或多个块无法访问（即 LSE），或者存放着错误的内容（即损坏）。因此，访问一块看起来正常的磁盘时，它偶尔可能在读写某个块时返回错误（非静默的部分故障），偶尔也可能直接返回错误的数据（静默的部分故障）。

这两类故障都不太常见，但到底有多不常见呢？图 45.1 汇总了 Bairavasundaram 的两项研究中的部分发现 [B+07, B+08]。

这项研究持续了大约 3 年，涉及超过 150 万块磁盘驱动器（disk drive）。图中给出的是研究期间至少出现过一次 LSE 或块损坏（block corruption）的驱动器所占的百分比。图中还把结果细分为“廉价”驱动器（通常是 SATA 驱动器）和“高价”驱动器（通常是 SCSI 或光纤通道驱动器）。可以看到，虽然买更好的驱动器能降低这两类问题的发生频率（大约降低一个数量级），但它们发生得仍然足够频繁，需要你仔细考虑如何在存储系统中处理它们。

关于 LSE 的其他一些发现：

- 有不止一个 LSE 的高价驱动器，和廉价驱动器一样容易出现更多错误
- 对大多数驱动器来说，年错误率在第二年上升
- LSE 的数量随磁盘容量增大而增加
- 大多数有 LSE 的磁盘，LSE 少于 50 个
- 有 LSE 的磁盘更容易出现更多的 LSE
- 存在显著的空间局部性（spatial locality）和时间局部性（temporal locality）
- disk scrubbing 很有用（大多数 LSE 都是这样发现的）

关于损坏的一些发现：

- 在同一类驱动器中，不同型号的损坏几率差别很大
- 使用年限的影响因型号而异
- 工作负载（workload）和磁盘容量对损坏几乎没有影响
- 大多数出现损坏的磁盘只有少数几处损坏
- 在同一块磁盘内，或者 RAID 中的多块磁盘之间，损坏并不是相互独立的
- 存在空间局部性，也有一定的时间局部性
- 与 LSE 之间有微弱的相关性

要进一步了解这些故障，你大概应该读读原始论文 [B+07, B+08]。但希望要点已经很清楚了：如果你真的想构建一个可靠的存储系统，就必须加入检测 LSE 和块损坏（block corruption）并从中恢复的机制。

45.2 处理 latent sector error

了解了这两种新的部分故障模式，现在来看看能拿它们怎么办。先处理两者中较容易的那个：latent sector error。

关键问题：如何处理 latent sector error
存储系统应该如何处理 latent sector error？处理这种形式的部分故障需要多少额外的机制？

事实证明，latent sector error 处理起来相当直接，因为按照定义，它们很容易被检测到。存储系统访问某个块时，如果磁盘返回错误，它只需借助自己现有的冗余（redundancy）机制返回正确的数据即可。例如，在镜像 RAID 中，系统应该访问另一个副本；在基于奇偶校验（parity）的 RAID-4 或 RAID-5 系统中，系统应该利用奇偶校验组中的其他块重建这个块。因此，像 LSE 这样容易检测到的问题，借助标准的冗余机制就能轻松恢复。

多年来，LSE 越来越普遍，这影响了 RAID 的设计。在 RAID-4/5 系统中，当整盘故障和 LSE 同时出现时，会产生一个特别有意思的问题。具体来说，当整块磁盘失效时，RAID 会尝试**重建**（reconstruct）这块磁盘（比如重建到一块热备盘（hot spare）上），方法是读遍奇偶校验组（parity group）中所有其他磁盘，重新计算出缺失的值。如果在重建过程中，其他任何一块磁盘上遇到了 LSE，我们就有麻烦了：重建无法成功完成。

为了解决这个问题，有些系统增加了一层额外的冗余。例如，NetApp 的 **RAID-DP** 相当于有两块奇偶校验盘（parity disk），而不是一块 [C+04]。在重建过程中发现 LSE 时，额外的奇偶校验可以帮助重建缺失的块。和往常一样，这是有代价的：为每个条带（stripe）维护两个奇偶校验块（parity block）的开销更大；不过，NetApp 的 **WAFL** 文件系统采用日志结构（log-structured），在许多情况下减轻了这一开销 [HLM94]。剩下的代价是空间，也就是要为第二个奇偶校验块（parity block）多用一块磁盘。

45.3 检测损坏：校验和

现在我们来处理更具挑战性的问题，也就是由数据损坏引起的静默错误（silent fault）。当出现损坏、导致磁盘返回坏数据时，我们怎样才能防止用户拿到坏数据？

关键问题：如何在出现损坏时保持数据完整性
鉴于这类故障是静默的，存储系统能做些什么来检测损坏的发生？需要哪些技术？如何高效地实现它们？

和 latent sector error 不同，检测损坏才是关键问题。客户端怎么判断一个块已经坏了？一旦知道某个块坏了，恢复的方法就和以前一样：你需要有这个块的另一个副本（而且最好是没有损坏的！）。因此，我们这里重点讨论检测技术。

提示：天下没有免费的午餐

There's No Such Thing As A Free Lunch（天下没有免费的午餐），简称 TNSTAAFL，是一句古老的美国俗语，意思是：当你看似免费得到某样东西时，实际上很可能在为它付出某种代价。它源自过去的年代：小餐馆会打出免费午餐的广告来招揽顾客；你走进去以后才会发现，要得到这份“免费”午餐，必须买一杯或几杯酒。当然，这未必是坏事，尤其是如果你立志成为酒鬼（或者是个典型的本科生）。

校验和（checksum）是现代存储系统用来保持数据完整性（data integrity）的主要机制。校验和就是某个函数的计算结果：函数的输入是一块数据（比如一个 4 KB 的块），计算后得到一个反映数据内容的小摘要（比如 4 或 8 个字节），这个摘要就是校验和。这样做的目的，是让系统能够发现数据是否遭到了损坏或改动：把校验和与数据存在一起，以后访问数据时，再确认数据当前的校验和与当初存下的值是否一致。

常用的校验和函数

计算校验和的函数有很多种，它们在强度（即保护数据完整性（data integrity）的能力有多强）和速度（即计算有多快）上各不相同。这里出现了系统中常见的一种权衡：通常，你得到的保护越多，代价就越高。天下没有免费的午餐。

有一种简单的校验和函数基于异或（XOR）：把要计算校验和的数据块（data block）逐段做 XOR，最终得到一个代表整个块 XOR 结果的值，这就是校验和。

为了说得更具体些，假设我们要在一个 16 字节的块上计算 4 字节的校验和；这个块当然太小，不能真正作为磁盘扇区或块，但用来举例足够了。这 16 个数据字节用十六进制表示如下：

```
365e c4cd ba14 8a92 ecef 2c3a 40be f666
```

如果用二进制来看，就是这样：

```
0011 0110 0101 1110  1100 0100 1100 1101
1011 1010 0001 0100  1000 1010 1001 0010
1110 1100 1110 1111  0010 1100 0011 1010
0100 0000 1011 1110  1111 0110 0110 0110
```

由于我们把数据按每行 4 个字节排好了，很容易看出校验和的结果是什么：对每一列做 XOR，就得到最终的校验和值：

```
0010 0000 0001 1011  1001 0100 0000 0011
```

结果用十六进制表示是 0x201b9403。

XOR 是一种还算合理的校验和，但有它的局限。例如，如果参与计算的两个单元中，同一位置上的位都发生了变化，校验和就检测不到这种损坏。正因如此，人们研究了其他的校验和函数。

 © 2008–23, ARPACI-DUSSEAU

另一种基本的校验和函数是加法。这种方法的优点是速度快：计算时只需对数据的每一段做二进制补码加法，忽略溢出即可。它能检测出数据的许多变化，但如果数据发生了移位之类的变化，效果就不好了。

一种稍微复杂一点的算法叫作 **Fletcher 校验和**（Fletcher checksum）；你大概猜到了，它以发明者 John G. Fletcher 的名字命名 [F82]。它计算起来很简单，需要算出两个校验字节 $s1$ 和 $s2$。具体来说，假设块 D 由字节 $d1...dn$ 组成；$s1$ 的定义是：$s1 = (s1 + d_i) \bmod 255$（对所有 d_i 计算）；$s2$ 则是：$s2 = (s2 + s1) \bmod 255$（同样对所有 d_i 计算）[F04]。Fletcher 校验和（Fletcher checksum）几乎和 CRC（见下文）一样强，能检测出所有单比特错误、双比特错误，以及许多突发错误 [F04]。

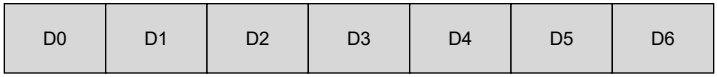
最后一种常用的校验和叫作**循环冗余校验**（cyclic redundancy check，**CRC**）。假设你想对数据块 D 计算校验和。你只需把 D 当成一个很大的二进制数（毕竟它只是一串位），然后用它除以一个约定好的值（k）。这个除法的余数就是 CRC 的值。事实证明，这种二进制取模运算可以实现得相当高效，这也是 CRC 在网络领域同样流行的原因。更多细节请参阅其他资料 [M13]。

无论用哪种方法，显然都不存在完美的校验和：两个内容不同的数据块可能有相同的校验和，这称为**碰撞**（collision）。这一点应该不难理解：毕竟，计算校验和就是把很大的东西（例如 4 KB）变成小得多的摘要（例如 4 或 8 个字节）。因此，选择好的校验和函数时，我们要找的是一个既能把碰撞的几率降到最低、又容易计算的函数。

校验和的布局

你已经大致了解了如何计算校验和，接下来我们分析如何在存储系统中使用校验和。首先要解决的问题是校验和的布局，也就是说，校验和应该怎样存放在磁盘上？

最基本的方法，就是为每个磁盘扇区或块存一个校验和。数据块 D 的校验和记作 $C(D)$。于是，没有校验和时，磁盘布局是这样的：

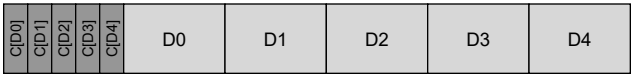


有了校验和之后，布局中每个块都多了一个校验和：



校验和通常很小（例如 8 字节），而磁盘只能以扇区（512 字节）或其整数倍为单位写入，那么上面这种布局该怎么实现呢？驱动器厂商采用的一种解决办法，是把驱动器格式化成 520 字节的扇区；每个扇区多出来的 8 个字节可以用来存放校验和。

如果磁盘没有这种功能，文件系统就必须想办法把校验和打包存放到 512 字节的块中。一种可行的做法如下：



 © 2008–23, ARPACI-DUSSEAU

在这种方案中，n 个校验和一起存放在一个扇区里，后面跟着 n 个数据块，再后面是下 n 个块的另一个校验和扇区，依此类推。这种方法的好处是适用于所有磁盘，但效率可能较低；例如，如果文件系统想覆写块 $D1$，它就必须读入包含 $C(D1)$ 的校验和扇区（checksum sector），更新其中的 $C(D1)$，然后写出校验和扇区和新的数据块 $D1$（因此要一次读和两次写）。前一种方法，即每个扇区存一个校验和，只需执行一次写入。

45.4 使用校验和

确定了校验和的布局，就可以来看看怎样使用校验和了。读取块 D 时，客户端（即文件系统或存储控制器）会同时从磁盘读出它的校验和 $C_s(D)$，称为**存储的校验和**（stored checksum），下标 s 就是这么来的。然后，客户端对取回的块 D 计算校验和，我们称之为**计算出的校验和**（computed checksum）$C_c(D)$。这时，客户端比较存储的校验和（stored checksum）与计算出的校验和（computed checksum）；如果两者相等（即 $C_s(D) == C_c(D)$），数据很可能没有损坏，因此可以安全地返回给用户。如果两者不相等（即 $C_s(D)\ != C_c(D)$），就说明数据自存储之后发生了变化，因为存储的校验和（stored checksum）反映的是当时数据的值。这说明遇到了损坏，而校验和帮我们发现了它。

出现损坏时，自然要问：该怎么办？如果存储系统有冗余副本（redundant copy），答案很简单：试着改用它。如果存储系统没有这样的副本，可能的答案就是返回一个错误。无论哪种情况，都要认识到，检测损坏并不是灵丹妙药；如果没有其他办法拿到未损坏的数据，你就只能自认倒霉了。

45.5 一个新问题：misdirected write

上面描述的基本方案，在块损坏（block corruption）的一般情况下效果很好。然而，现代磁盘有几种不寻常的故障模式，需要不同的解决方案。

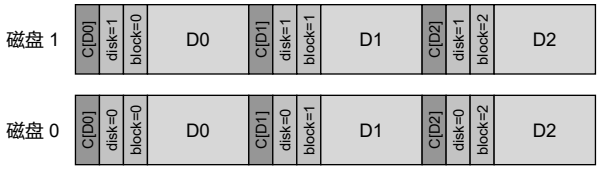
我们关注的第一种故障模式叫作 **misdirected write**。它出现在磁盘和 RAID 控制器中：数据正确地写到了磁盘上，只是写到了错误的位置。在单磁盘系统中，这意味着磁盘没有把块 D_x 写到（期望的）地址 x，而是写到了地址 y（因此“损坏”了 D_y）；此外，在多磁盘系统中，控制器还可能没有把 $D_{i,x}$ 写到磁盘 i 的地址 x，而是写到了另一块磁盘 j 上。于是我们的问题是：

> 关键问题：如何处理 misdirected write
> 存储系统或磁盘控制器应该如何检测 misdirected write？校验和需要具备哪些额外的特性？

答案不出所料，很简单：在每个校验和里多加一点信息。在这种情况下，加入一个**物理标识符**（physical identifier，**physical ID**）很有帮助。例如，如果存储的信息现在不仅包含校验和 $C(D)$，还包含块所在的磁盘号和扇区号，客户端就很容易判断某个位置上存放的是不是正确的信息。具体来说，如果客户端要读取磁盘 10 上的块 4（$D_{10,4}$），存储的信息中就

 © 2008–23, ARPACI-DUSSEAU

应该包含这个磁盘号和扇区偏移。如果信息对不上，说明发生了 misdirected write，损坏就被检测出来了。下面是在一个双磁盘系统上加入这些信息后的样子。注意，这张图和之前的那些图一样，并没有按比例画，因为校验和通常很小（例如 8 字节），而块要大得多（例如 4 KB 或更大）：



从盘上的格式可以看出，磁盘上的冗余现在相当多：每个块里都重复存放了磁盘号，块自己的偏移量（offset）也存放在块旁边。有冗余信息并不奇怪，冗余正是检测错误（这里的情况）和从错误中恢复（其他情况）的关键。对完美的磁盘来说，这点额外信息并不是必需的，但一旦出了问题，它能帮上大忙。

45.6 最后一个问题：lost write

可惜的是，misdirected write 并不是我们要处理的最后一个问题。具体来说，一些现代存储设备还有一个叫作 **lost write** 的问题：设备告诉上层某次写入已经完成，但数据实际上从未持久化，盘上留下的还是块的旧内容，而不是更新后的新内容。

这里显而易见的问题是：上面那些校验和策略（例如基本的校验和或物理标识）能帮助检测 lost write 吗？可惜，答案是否定的：旧块很可能有匹配的校验和，而上面用到的物理 ID，也就是磁盘号和块偏移量，也是正确的。于是，我们最后的问题是：

> 关键问题：如何处理 lost write
> 存储系统或磁盘控制器应该如何检测 lost write？校验和需要具备哪些额外的特性？

有许多可能的解决方案可以帮上忙 [K+08]。一种经典的方法 [BS04] 是执行 **write verify**，也叫 **read-after-write**：写入之后立即把数据读回来，系统就能确保数据确实到达了盘面。然而，这种方法相当慢，完成一次写入所需的 I/O 数量翻了一倍。

有些系统在别处再加一份校验和来检测 lost write。例如，Sun 的 **Zettabyte File System**（**ZFS**）在每个文件系统 inode 和间接块（indirect block）中，为文件包含的每个块都存了一个校验和。这样，即使对数据块本身的写入丢失了，inode 中的校验和也不会与旧数据相符。只有对 inode 和对数据的写入同时丢失时，这种方案才会失效，这种情况不太可能发生（但很不幸，是有可能的！）。

45.7 Scrubbing

讨论了这么多，你可能会想：这些校验和到底是什么时候被检查的？当然，应用程序访问数据时会检查一部分，但大多数数据很少被访问，也就一直没人检查。对可靠的存储系统

来说，没有检查过的数据是个问题，因为位衰减（bit rot）最终可能会影响某份数据的所有副本。

为了解决这个问题，许多系统采用了各种形式的 **disk scrubbing** [K+08]。磁盘系统定期读遍系统中的每一个块，检查校验和是否仍然有效，就能降低某个数据项的所有副本都损坏的几率。典型的系统会每晚或每周安排一次扫描。

45.8 校验和的开销

在结束本章之前，我们来讨论一下用校验和保护数据的一些开销。和计算机系统中常见的情况一样，开销分为两种：空间和时间。

空间开销有两种形式。第一种在磁盘（或其他存储介质）本身上：每个存储的校验和（stored checksum）都要占用磁盘空间，这些空间不能再用来存放用户数据。典型的比例可能是每 4 KB 数据块配一个 8 字节的校验和，盘上空间开销为 0.19%。

第二种空间开销出现在系统的内存中。访问数据时，内存中除了数据本身，现在还必须为校验和留出空间。不过，如果系统只是检查校验和，检查完就丢弃，这种开销是短暂的，不必太担心。只有当校验和保留在内存中（以额外防护内存损坏 [Z+13]）时，这点小开销才会显现出来。

虽然空间开销很小，但校验和带来的时间开销可能相当明显。至少，CPU 必须对每个块计算校验和：存储数据时要算，以确定存储的校验和（stored checksum）的值；访问数据时要再算一次，并与存储的校验和比较。许多使用校验和的系统（包括网络协议栈（protocol stack））都采用了一种降低 CPU 开销的方法：把数据复制和计算校验和合并成一个流水线式的操作。复制本来就免不了（例如把数据从内核页缓存（page cache）复制到用户缓冲区），顺便算校验和就很划算。

除了 CPU 开销，有些校验和方案还会带来额外的 I/O 开销：一是校验和与数据分开存放时，访问校验和需要额外的 I/O；二是后台 scrubbing 需要额外的 I/O。前者可以靠好的设计减少；后者可以调节，比如控制 scrubbing 在什么时候进行，以限制它的影响。深夜时分，大多数（不是所有！）勤奋的员工都已经上床睡觉了，这可能是执行 scrubbing、提高存储系统健壮性的好时机。

45.9 小结

我们讨论了现代存储系统中的数据保护（data protection），重点是校验和的实现和使用。不同的校验和防护不同类型的故障；随着存储设备的发展，毫无疑问还会出现新的故障模式。也许这些变化会迫使学术界和工业界重新审视其中一些基本方法，或者干脆发明全新的方法。时间会告诉我们答案。或者不会。时间就是这么有意思。

参考文献

[B+07] "An Analysis of Latent Sector Errors in Disk Drives" by L. Bairavasundaram, G. Goodson, S. Pasupathy, J. Schindler. SIGMETRICS '07, San Diego, CA. *The first paper to study latent sector errors in detail. The paper also won the Kenneth C. Sevcik Outstanding Student Paper award, named after a brilliant researcher and wonderful guy who passed away too soon. To show the OSTEP authors it was possible to move from the U.S. to Canada, Ken once sang us the Canadian national anthem, standing up in the middle of a restaurant to do so. We chose the U.S., but got this memory.*

[B+08] "An Analysis of Data Corruption in the Storage Stack" by Lakshmi N. Bairavasundaram, Garth R. Goodson, Bianca Schroeder, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST '08, San Jose, CA, February 2008. *The first paper to truly study disk corruption in great detail, focusing on how often such corruption occurs over three years for over 1.5 million drives.*

[BS04] "Commercial Fault Tolerance: A Tale of Two Systems" by Wendy Bartlett, Lisa Spainhower. IEEE Transactions on Dependable and Secure Computing, Vol. 1:1, January 2004. *This classic in building fault tolerant systems is an excellent overview of the state of the art from both IBM and Tandem. Another must read for those interested in the area.*

[C+04] "Row-Diagonal Parity for Double Disk Failure Correction" by P. Corbett, B. English, A. Goel, T. Grcanac, S. Kleiman, J. Leong, S. Sankar. FAST '04, San Jose, CA, February 2004. *An early paper on how extra redundancy helps to solve the combined full-disk-failure/partial-disk-failure problem. Also a nice example of how to mix more theoretical work with practical.*

[F04] "Checksums and Error Control" by Peter M. Fenwick. Copy available online here: http://www.ostep.org/Citations/checksums-03.pdf. *A great simple tutorial on checksums, available to you for the amazing cost of free.*

[F82] "An Arithmetic Checksum for Serial Transmissions" by John G. Fletcher. IEEE Transactions on Communication, Vol. 30:1, January 1982. *Fletcher's original work on his eponymous checksum. He didn't call it the Fletcher checksum, rather he just didn't call it anything; later, others named it after him. So don't blame old Fletch for this seeming act of braggadocio. This anecdote might remind you of Rubik; Rubik never called it "***Rubik's cube***"; rather, he just called it "my cube."*

[HLM94] "File System Design for an NFS File Server Appliance" by Dave Hitz, James Lau, Michael Malcolm. USENIX Spring '94. *The pioneering paper that describes the ideas and product at the heart of NetApp's core. Based on this system, NetApp has grown into a multi-billion dollar storage company. To learn more about NetApp, read Hitz's autobiography "How to Castrate a Bull" (which is the actual title, no joking). And you thought you could avoid bull castration by going into CS.*

[K+08] "Parity Lost and Parity Regained" by Andrew Krioukov, Lakshmi N. Bairavasundaram, Garth R. Goodson, Kiran Srinivasan, Randy Thelen, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST '08, San Jose, CA, February 2008. *This work explores how different checksum schemes work (or don't work) in protecting data. We reveal a number of interesting flaws in current protection strategies.*

[M13] "Cyclic Redundancy Checks" by unknown. Available: http://www.mathpages.com/home/kmath458.htm. *A super clear and concise description of CRCs. The internet is full of information, as it turns out.*

[P+05] "IRON File Systems" by V. Prabhakaran, L. Bairavasundaram, N. Agrawal, H. Gunawi, A. Arpaci-Dusseau, R. Arpaci-Dusseau. SOSP '05, Brighton, England. *Our paper on how disks have partial failure modes, and a detailed study of how modern file systems react to such failures. As it turns out, rather poorly! We found numerous bugs, design flaws, and other oddities in this work. Some of this has fed back into the Linux community, thus improving file system reliability. You're welcome!*

[RO91] "Design and Implementation of the Log-structured File System" by Mendel Rosenblum and John Ousterhout. SOSP '91, Pacific Grove, CA, October 1991. *So cool we cite it again.*

[S90] "Implementing Fault-Tolerant Services Using The State Machine Approach: A Tutorial" by Fred B. Schneider. ACM Surveys, Vol. 22, No. 4, December 1990. *How to build fault tolerant services. A must read for those building distributed systems.*

[Z+13] "Zettabyte Reliability with Flexible End-to-end Data Integrity" by Y. Zhang, D. Myers, A. Arpaci-Dusseau, R. Arpaci-Dusseau. MSST '13, Long Beach, California, May 2013. *How to add data protection to the page cache of a system. Out of space, otherwise we would write something…*

 © 2008–23, ARPACI-DUSSEAU

作业（模拟）

在这个作业中，你要用 `checksum.py` 研究校验和的各个方面。

问题

1. 首先，不带任何参数运行 `checksum.py`。计算加法校验和、基于 XOR 的校验和以及 Fletcher 校验和（Fletcher checksum）。用 `-c` 检查你的答案。
2. 现在做同样的事，但把种子（`-s`）改成不同的值。
3. 有时，加法校验和与基于 XOR 的校验和会得出相同的结果（例如，数据值全为零时）。你能传入一个 4 字节的数据值（用 `-D` 选项，例如 `-D a,b,c,d`），它不全为零，却让加法校验和与基于 XOR 的校验和的值相同吗？一般来说，什么时候会出现这种情况？用 `-c` 选项检查你是否正确。
4. 现在传入一个 4 字节的值，你知道它会让加法和 XOR 得出不同的校验和值。一般来说，什么时候会出现这种情况？
5. 用模拟器计算两次校验和（每次用一组不同的数）。两串数字应该不同（例如第一次用 `-D a1,b1,c1,d1`，第二次用 `-D a2,b2,c2,d2`），但要得出相同的加法校验和。一般来说，在数据值不同的情况下，加法校验和什么时候会相同？用 `-c` 选项检查你的具体答案。
6. 现在对 XOR 校验和做同样的事。
7. 现在我们来看一组特定的数据值。第一组是：`-D 1,2,3,4`。对这组数据，各种校验和（加法、XOR、Fletcher）分别是多少？再把它和对 `-D 4,3,2,1` 计算这些校验和的结果比较一下。你注意到这三种校验和有什么特点？Fletcher 和另外两种相比怎么样？一般来说，Fletcher 为什么比简单的加法校验和之类的“更好”？
8. 没有完美的校验和。给定一个你自己选的输入，你能找到其他数据值，得出相同的 Fletcher 校验和（Fletcher checksum）吗？一般来说，什么时候会出现这种情况？从一个简单的数据串开始（例如 `-D 0,1,2,3`），看看能否替换其中一个数，却得到相同的 Fletcher 校验和（Fletcher checksum）。和往常一样，用 `-c` 检查你的答案。

 © 2008–23, ARPACI-DUSSEAU

作业（编程）

在这部分作业中，你要自己写一些代码来实现各种校验和。

问题

1. 写一个简短的 C 程序（名为 `check-xor.c`），对输入文件计算基于 XOR 的校验和，并把校验和打印出来。用一个 8 位的 `unsigned char` 来存放（一个字节的）校验和。制作一些测试文件，看看它是否按预期工作。
2. 现在写一个简短的 C 程序（名为 `check-fletcher.c`），对输入文件计算 Fletcher 校验和（Fletcher checksum）。同样，测试一下你的程序，看看它是否能工作。
3. 现在比较两者的性能：哪个更快？输入文件的大小变化时，性能如何变化？在程序内部调用 `gettimeofday` 来计时。如果你关心性能，应该用哪一个？如果关心检错能力呢？
4. 了解一下 16 位 CRC，然后实现它。用许多不同的输入测试它，确保它能正常工作。和简单的 XOR、Fletcher 相比，它的性能如何？检错能力呢？
5. 现在构建一个工具（`create-csum.c`），对文件的每个 4 KB 块计算一个单字节的校验和，并把结果记录到一个输出文件中（在命令行中指定）。再构建一个相关的工具（`check-csum.c`），它读取一个文件，对每个块计算校验和，并把结果和存放在另一个文件中的校验和比较。如果有问题，程序应该打印出文件已经损坏的信息。手动破坏文件来测试这个程序。

 © 2008–23, ARPACI-DUSSEAU